

DOCUMENTO DE ARQUITECTURA DE SOFTWARE

DGS

Especificación Técnica de Estructuración de Proyecto, Flujo Lógico y Diseño de Arquitectura

Para uso de:	Cuerpo de Profesores y Colaboradores
Fecha de Creación:	Junio, 2026
Estado:	Aprobado - Configuración de Proyecto Inicializada

1. Introducción y Contexto del Proyecto

El proyecto DGS surge como una modernización educativa para la facultad. Anteriormente, los alumnos utilizaban una aplicación de escritorio para comprender los fundamentos de la lógica de programación. Dicho entorno se basaba en la construcción de algoritmos mediante diagramas de bloques (conceptualmente similares a los diagramas de flujo tradicionales, pero con una representación visual más amplia y acotada al flujo de ejecución).

La nueva propuesta migra esta lógica a una aplicación web moderna (SPA), permitiendo a los estudiantes acceder de manera ágil desde cualquier navegador sin necesidad de instalaciones locales. Toda la persistencia y ejecución de algoritmos se resuelve de forma local en el navegador del alumno. Esto reduce los costos de despliegue a cero y acelera el desarrollo inicial del prototipo funcional.

Los 6 Bloques Lógicos Fundamentales (Modelo Conceptual)

El estudiante compone sus soluciones combinando 6 elementos lógicos que representan las estructuras de datos y control básicas presentes en cualquier lenguaje de programación:

- **1. Input (Entrada):** Permite solicitar datos al usuario para guardarlos en una variable. Ej: `|input: x|`.
- **2. Output (Salida):** Muestra en pantalla un texto literal o el valor de una variable. Ej: `|output: x|` o `|output: "hola"|`.
- **3. Asignación:** Inicializa o actualiza el valor de una variable. Ej: `|asigna: x = 5|` o `|asigna: c = c + 1|`.
- **4. If-Else (Condicional):** Define una condición lógica. Se divide visualmente en el camino verdadero y camino falso, ejecutando el bloque que corresponda.
- **5. Ciclo Mientras (While):** Repite los bloques en su interior de forma continua mientras se cumpla la condición lógica.
- **6. Ciclo Para (For):** Bucle iterador que ejecuta los bloques en su interior desde un límite inferior (a) hasta un límite superior (b).

2. Estructura de Proyecto Limpia

El proyecto se organiza de manera simplificada en una estructura de frontend limpia. Hemos removido todas las dependencias externas innecesarias para el arranque, permitiendo que el código fuente principal resida directamente en el directorio raíz.

La estructura actual de directorios en el repositorio es la siguiente:

```
DGS/
├─ src/
│   └─ components/      # Lienzo interactivo, barra de bloques, panel de código,
consola
│   └─ core/            # Tipos de AST, parser y compiladores (C / JS)
│   └─ hooks/           # Hooks de control de parseo y ejecución de código
│   └─ utils/           # Módulos utilitarios (compresión URL para compartir)
│   └─ App.tsx          # Layout principal y estado de la aplicación
│   └─ main.tsx         # Punto de entrada de React
├─ public/              # Recursos estáticos del navegador
├─ docs/                # Documentación del proyecto (este PDF)
├─ package.json         # Dependencias y scripts de Vite
└─ .gitignore           # Exclusiones de Git
```

3. Patrones y Estructuras Técnicas

El cliente React encapsula toda la funcionalidad crítica sin depender de servidores. Los puntos clave son:

- **Compresión y URL Sharing:** El sistema admite descargar los programas en archivos planos `.json` o comprimirlos usando `lz-string` para generar un enlace autodescriptivo que carga el diagrama directamente desde la URL.
- **Tipado Estricto de Bloques:** Los tipos en TypeScript mapean la estructura de datos recursiva (AST) para facilitar la renderización dinámica en el Canvas de la aplicación y la validación en tiempo de ejecución.
- **Interpreter Engine:** Ubicado en `src/core/interpreter/runner.ts`, el motor de ejecución evalúa paso a paso el programa y proporciona asincronicidad usando la biblioteca `js-interpreter`, pausándose cuando necesita la respuesta del alumno a un bloque input.